

Goldsmiths, University of London

TasteMender: A stateless music recommendation API

CM3070 Computer Science Final Project - Report

<https://github.com/RadValentin/CM3070-FP-Music-Recommendation>

Valentin Radulescu
22.09.2025

Table of Contents

Table of Contents	1
List of Figures	4
List of Tables.....	5
Introduction (863/1000 words)	6
Literature Review (2433/2500 words)	8
The Evolution of Recommender Systems.....	8
Information Filtering and Expert Systems	8
The Introduction of Collaborative Filtering.....	8
Content-Based Filtering.....	9
Hybrid Recommendation Approaches	10
The Deep Learning Era	10
Deep Learning Models	11
Embeddings and Graph-Based Approaches	11
Music Databases and Datasets	11
Million Song Dataset	11
FMA (Free Music Archive).....	12
Datasets on Kaggle.....	12
Spotify API	13
Discogs and MusicBrainz	13
AcousticBrainz.....	13
Conclusion	14
Design (1949/2000 words)	15
Domain and Users	15
System Architecture	15
Data Layer	16
Application Layer	17
Presentation Layer	19
Prototype and Design Evolution	20
Project name.....	21
Work Plan	21

Evaluation plan	22
Implementation (2121/2500 words).....	24
Overview.....	24
Data Ingest Pipeline	24
Schema.....	24
File Loading	25
Data Extraction	25
Duplicate Handling	26
Database Construction	26
Performance metrics.....	27
Recommendation Engine	27
Feature Selection	27
Pre-filtering.....	28
Similarity Computation.....	28
Output	29
API Layer.....	30
API Documentation	31
Search	31
Recommendation Endpoint	32
Front-End Integration	33
Running the project	34
Evaluation (1152/2500 words).....	35
Performance Evaluation	35
Recommendation Quality	35
User Testing	36
Critical Analysis	36
Dataset Limitations	37
Future Work	37
Conclusion (528/1000 words)	38
References	40
Appendix.....	42
Appendix A.1 - Gantt Chart.....	42

Appendix A.2 - API Endpoints.....	43
Appendix A.3 - AcousticBrainz High-Level JSON	44
Appendix A.4 - Running the Project	45

List of Figures

Figure 1 - Entity Relationship Diagram.....	17
Figure 2 - POST request body for recommend endpoint.....	18
Figure 3 - Recommend endpoint simplified response.....	19
Figure 4 - Prototype feature extraction and similarity.....	20
Figure 5 - Lyrics based recommendations for “Master of Puppets” by Metallica	21
Figure 6 - Graph of the time spanned by each phase	22
Figure 7 - Code repository structure.....	24
Figure 8 - Album view with "art" action	30
Figure 9 - Swagger docs for recommend endpoint	31
Figure 10 - Django QuerySet and the SQL it translates to	32
Figure 11 - Indexes on Track model	32
Figure 12 - The player interface.....	34
Figure 13 - Gantt Chart for Project Development.....	42
Figure 14 - Example .env file	46

List of Tables

Table 1 - Performance vs dataset size	27
Table 2 - Features selected and their accuracy scores	28
Table 3 - How dimensionality affects results.....	29
Table 4 - How MIREX features affect results.....	30

Introduction (863/1000 words)

Over the past two decades we've seen a slow but steady change in the way people consume and interact with media. As more aspects of our daily lives have shifted over from the physical world to online so did the way we engage with them evolve. The previously monolithic landscape where major names dominated the collective consciousness, due to their mass-market appeal, gradually became fragmented. As early as 2009 journalists were declaring the death of the monoculture (Murray, 2009), a trend that was being accelerated by the rise of digital platforms.

When it came to music, the dominant gatekeepers: MTV, radio stations, billboard charts, that previously served to inform preferences were starting to be replaced by digital platforms (e.g. Spotify, SoundCloud, YouTube) which democratized content creation and distribution (Anderson, 2006). Anyone could become a creator in this new environment and as the volume and diversity of available content increased, the relationship between artists and audiences shifted in a fundamental way. In a saturated market, artists found it hard to stand out and in order to gain visibility and traction, many creators began tailoring their sound to appeal to very specific tastes, niche audiences and subcultures.

On the other hand, listeners grew accustomed to having their very specific tastes met in a way that the traditional mass-market model couldn't fulfil. The modern listener seeks out not just genres but moods, aesthetics and emotional states. Music is to him an extension of his own identity. While this dynamic allows for a richer and more personalized listening experience it also introduces a new challenge: content overload. Even though the specific song or mood he's looking for exists, it's difficult to find. It's no longer a matter of what's trending but of how to discover – a challenge which requires a new approach (Qin, 2013).

Music recommendation systems are tools needed to bridge the gap between audience preference fragmentation and an oversaturated overspecialized content landscape. They typically fall into three categories: collaborative filtering, content-based filtering, or hybrid approaches, and are widely used nowadays. We can often encounter the effects of their use, for example, on YouTube, lo-fi hip hop study or chill playlists consistently gain millions of views, so do electro swing artists on Spotify. In both, and many other cases, the success of niche content isn't accidental but the result of personalized recommendation algorithms that made predictions based on users' preferences, history, habits, etc. (Celma, 2010).

Recommender systems are nowadays an essential part of navigating the modern media ecosystem, being widely used by most major platforms. The need for making more accurate predictions has created a scenario where complex user profiles are being stored on centralized servers. Simply put, the more data is available about the user, the

more accurate the prediction – a practice which invites questions about consent, data ownership, privacy and legal boundaries (Asad et al., 2023).

In response to the concerns outlined above, this project explored a different approach: a stateless music recommendation system titled *TasteMender*, which suggests which track should be played next based on user provided preferences and filters. Unlike popular music services which rely heavily on user profiling and data collection, this project will operate as a stateless REST API which enforces a privacy-friendly approach. This means that the system does not store user data, track users or require the creation of persistent user profiles.

The project is based on the “*Project Idea 2: NextTrack: A music recommendation API*” template which builds upon the information taught in the “*CM3035 Advanced Web Development*” module. The implementation will consist of a back-end API developed in Django and a front-end interface built with React that will demonstrate the functionality and allow for user testing. The recommendation engine itself will be powered by a combination of rule-based filtering and a basic machine learning model applied to open music metadata.

The core motivation behind TasteMender is to explore an alternative to how popular music services handle track recommendations. Platforms like Spotify and YouTube Music typically rely heavily on user data and give opaque recommendations with very limited user control. While the results are often personalized, there’s also a lack of transparency going on. Users are not typically aware of how recommendations are made and have limited ability to influence the logic behind them. In the long-term, repeated exposure to certain favourite tracks tends to create a sort of bubble that users get stuck in, where new music discovery becomes increasingly difficult. This problem is exacerbated when multiple users share the same account, introducing ‘preference pollution’: tracks liked by the other user will keep getting recommended seemingly at random (e.g. nursery songs in a household with a baby, guilty favourites, etc.).

TasteMender aims to address these challenges by offering a transparent, user-controlled, and stateless recommendation experience. The system doesn’t rely on implicit data about the user, instead the user provides at front a list of liked tracks and sets optional filters (genre, mood, willingness-to-explore). The API then responds with one or more new tracks based solely on these inputs without needing to track user history or behaviour beyond the current session. Parameters such as randomness or willingness-to-explore will determine how predictable or adventurous the recommendations should be.

Literature Review (2433/2500 words)

This literature review explores the evolution of music recommendation systems from the early days of computing to modern times to inform the design of TasteMender, a privacy-first stateless music recommendation system. It also looks at concrete examples of how recommendation systems work, data sources that could support such a system and relevant algorithms.

The Evolution of Recommender Systems

Information Filtering and Expert Systems

The idea of filtering an information stream to remove unwanted data in order to present the most relevant parts to a human user predates the web and can be traced back to the early days of computing. Information retrieval (IR) systems were used in the 1960s and 1970s in libraries and scientific databases to return documents based on keyword queries (Salton, 1971). These systems worked by representing both the documents and queries as numerical vectors in a multi-dimensional space. Natural language processing (NLP) techniques like Term Frequency-Inverse Document Frequency (TF-IDF) were used to convert textual data into vectors. The documents were then ranked based on their proximity to the query in vector space, typically using cosine similarity (Salton & Buckley, 1988). This way the most relevant ones could be presented to the user.

In the 1980s the field of artificial intelligence was heavily focused on researching knowledge-based systems could solve complex problems by through automated reasoning applied to a base of knowledge. The first KBSs relied on an explicit set of rules being defined beforehand and were used to simulate the human decision-making process, that of someone who was an expert in the domain of knowledge (hence the name “expert systems”) (Felgenbaum, 1977). While not being recommender systems in the modern sense, they are early examples AI being used to automate decision making and they set the stage for later machine-learning approaches which replaced rigid rule-based reasoning with adaptive models capable of learning patterns from the data.

The Introduction of Collaborative Filtering

Recommender systems as we know them today started emerging in the early 1990s. They initially used a technique called collaborative filtering pioneered by projects such as GroupLens, a Usenet article recommendation engine, and Tapestry, a document recommendation engine for newsgroups (Melville & Sindhvani, 2011).

Collaborative Filtering (CF) systems make recommendations by focusing on the way users interact with items rather than the details of the items themselves. User preferences are inferred from the behaviour of others with similar tastes. These systems

removed the need for making explicit queries, a key limitation of IRs, by leveraging signals such as ratings, clicks or views. Netflix, Amazon and Spotify have all used variants of collaborative filtering to generate personalised recommendations (Koren et al., 2009).

The main selling points of CF systems are that they scale well with large datasets of user data, don't require detailed information about the item content as recommendations are generated purely from patterns in a user-item interaction matrix and can be used for almost any domain. However, they also have some very important weaknesses:

1. The *cold-start problem*: recommendations are hard to make for new platforms with little user data, for new users with little interaction history, or new items.
2. *Scalability costs*: large numbers of users and items require complex storage and processing solutions to aggregate data.
3. *Data sparsity*: generally, most users leave few reviews, even the most popular items will have a small number of reviews compared to the actual number of users who interacted with said item.

Content-Based Filtering

In response to the limitations of CFs, researchers developed Content-Based Filtering (CBF) systems. These systems recommend items by comparing the features of content that a user has previously liked with the features of candidate items. In a music setting, the features could be: genre, artist, tempo, lyrics, tags, etc. (Celma, 2010)

On the technical side, in CBF systems, data about items is represented as feature vectors. Simple features like tempo can be represented as a 1-dimensional value while for complex ones like lyrics, TF-IDF can be used to map word counts to N dimensions. Comparisons can be made between previously liked items and candidates by measuring the cosine similarity of their feature vectors.

Content-based approaches to making recommendations have two major strengths. First, they can handle new items more effectively than collaborative systems, since similarity is determined based on the intrinsic properties of the items themselves, not historical popularity. Second, they provide transparency, the system can explain its choices (e.g. this new song has similar lyrics or is in the same genre and one you previously listened to). The systems are also inherently more privacy-friendly than CFs since other user's data isn't required for making a choice.

The main drawbacks of CBF systems are:

1. *Over-specialisation*: In time they'll keep reinforcing the user's tastes thus reducing opportunities for discovering new items
2. Meaningful metadata is difficult to extract from items especially for subjective domains such as music (e.g. determine if two songs have the same "vibe")

For this project, CBF is particularly relevant because it aligns with the stateless and privacy-friendly goals of TasteMender.

Hybrid Recommendation Approaches

In the early 2000s, hybrid systems started being researched as a way to counteract the shortcomings of CFs and CBFs while at the same time leveraging their strengths (Burke, 2002). Hybrids are the natural evolution of earlier systems; they work by combining multiple recommendation techniques (CF + CBF) to produce more accurate and diverse results. Most notably, CF's ability to capture serendipitous discoveries and community-driven patterns can complement CBF's content-driven precision and cold-start resilience. This fusion works well in domains like music recommendation where rich content features (mood tags, genre tags, lyrics) can be combined with user interaction histories to accommodate the constant evolution of tastes.

For example, Netflix uses a hybrid recommendation approach which analyses user-item interaction data (ratings, views and watch times) to identify community trends and content metadata (genre, cast, and plot descriptions), to recommend items with similar attributes to those a user has previously enjoyed (Gomez-Urbe & Hunt, 2016).

The Deep Learning Era

The deep learning revolution which started in the mid-2010s also impacted recommender research. Collaborative filtering systems which relied on matrix factorisation were replaced by Neural Collaborative Filtering (NCF) systems, which use multi-layer perceptrons to identify nonlinear interactions between users and items (He et al., 2017).

Convolutional and recurrent neural networks have been used to extract rich metadata from raw audio or video sources which enabled music and multimedia recommenders to operate on richer feature spaces (Choi et al., 2016).

These systems and the models behind them promise greater accuracy and an increased ability to capture complex relationships in the data but they also have drawbacks: computational cost, lack of transparency regarding their reasoning and the risk of reinforcing biases captured in the training data.

Modern Approaches in Music Recommender Systems

We've seen how recommender systems have gradually evolved through the decades from collaborative filtering and content-based approaches toward methods that leverage advances in deep learning and neural networks. In the last decade developments in representation learning and graph-based modelling have allowed systems to capture complex patterns in data and scale them to billions of items. In this section we'll look at a few key concepts related to these developments as they apply to music recommendations.

Deep Learning Models

Deep learning models are used for handling many aspects of recommender systems, for example: Neural Collaborative Filtering for identifying arbitrary patterns in user interaction data or Sequential Pattern Learning for understanding chronological preferences as tastes shift.

In the music domain, deep learning has enabled direct extraction of metadata from raw signals. Choi et al. (2016) demonstrated that convolutional and recurrent neural networks can classify genre or mood directly from audio waveforms by producing embeddings (dense, low-dimensional vector representation of audio) which capture timbre, rhythm, and instrumentation.

Embeddings and Graph-Based Approaches

Embeddings can represent multiple aspects of data: audio features, lyrics or tags. They can be used to place songs in a joint semantic space (van den Oord et al., 2013) and naturally lend themselves to similarity search using cosine distance or nearest-neighbour indexing, enabling efficient retrieval even for very large datasets.

At industrial scale, Approximate Nearest Neighbour (ANN) algorithms provide a practical solution for making vector comparisons fast enough to be used in a real-time setting. Libraries such as FAISS (Johnson et al., 2021) and Annoy (Spotify, n.d.) enable the efficient retrieval of the top-k similar items in high dimensional space by trading a small loss in accuracy for major speed improvements. Annoy was developed by Spotify to power large-scale music similarity search while FAISS is heavily used by Meta Platforms to power multimedia document search.

Music Databases and Datasets

The datasets available to researchers significantly shape what kinds of systems can be developed. A large number of such datasets are openly available online that could support the implementation of a music recommendation system (“List of Online Music Databases,” 2025). In this section we’ll briefly cover the ones that are most relevant to the current project.

Million Song Dataset¹

A large-scale, publicly available dataset designed for music information retrieval (MIR) research. It was developed by The Echo Nest in collaboration with the Music Technology Group at Columbia University and released in 2011. Its goal was to provide a standardized resource for evaluating algorithms for music recommendation, genre classification and audio analysis (Bertin-Mahieux et al., 2011).

¹ <http://millionsongdataset.com/>

The dataset covers 1 million songs from 44,745 artists with a total size of around 300 GB. Rich metadata is provided, each track having a corresponding 55 features which cover: tempo, pitch, tags, terms, external identifiers, artist location, release year, etc. This level of detail makes it a prime candidate for the current project.

There is however a major downside when it comes to the age of the data as the last release was in 2012. What's more, a full copy of the dataset isn't readily available for download anymore from the official website at the time of writing.

FMA (Free Music Archive)²

Another popular dataset designed for MIR research. It was introduced in 2016 by researchers from École Polytechnique Fédérale de Lausanne (EPFL) and serves as a dump of the Free Music Archive website (freemusicarchive.org). It provides audio files along with metadata (tags, genre) and precomputed audio features (tempo, key loudness) for around 100k songs. Like the previous dataset, its age is a limiting factor as the last update was in 2017.

Datasets on Kaggle

Many small and apparently openly licensed datasets are freely available on Kaggle. We'll look at two as case studies to investigate their feasibility for the current project.

"Spotify Million Song Dataset"³, contrary to its name, contains records for around 57k songs. Each datapoint contains information about the artists, song name, URL and full-text lyrics. While sparse, the data could be used to match songs based on lyrics similarity using an NLP technique.

"Spotify 1 Million Tracks dataset"⁴ contains data on 1 million tracks released between 2000 and 2023. For each track a wide array of features were recorded: popularity, year, danceability, energy, key, loudness, mode, speechiness, acousticness, instrumentalness, liveness, valence, tempo, time signature, duration. The main downside here is the period covered: culturally relevant decades like the 1980s are missing so any recommendations informed by this dataset could feel limited, too recent or irrelevant.

One major limitation of these two datasets that's not readily apparent is the legality under which they're both licensed. On the surface, both are free to use for personal or commercial projects under the "Open Database License v1.0" and "CC0: Public Domain". However, according to the Spotify Developer Terms of Service⁵ the data retrieved from their API can be stored only if it supports the operation of an SDA (Spotify

² <https://github.com/mdeff/fma>

³ <https://www.kaggle.com/datasets/notshrirang/spotify-million-song-dataset>

⁴ <https://www.kaggle.com/datasets/amitanshjoshi/spotify-1million-tracks>

⁵ <https://developer.spotify.com/terms>

Developer Application) and must be updated regularly. Hosting a dump of Spotify data on Kaggle is outside these terms of service. The licensing issues represent a risk that makes using datasets from this platform a nonviable option. At best, they could be used to validate an idea before moving to a larger “canonical” dataset.

Spotify API

Aside from being a popular music platform, Spotify is also a good candidate for obtaining both audio metadata and precomputed features. The platform hosts more than 100 million songs and can be queried by developers using its API. For a recommendation system the access is rate to that API is limiting factor being capped to around 100 requests per hour⁶. It doesn't seem feasible to build an implementation that could compare a meaningful amount of track to generate a recommendation in real time. At best, we'd end up relying on Spotify's own recommendation service which defeats the purpose.

Discogs and MusicBrainz

These are massive open music databases that store information about millions of songs. They provide rich metadata which covers release information, artists, relationships between songs, cover art, etc. Both are community driven and updated continuously. In the scope of the current project the rich data they provide could be used as the base for driving recommendations.

AcousticBrainz

A collaborative, open-source initiative by the Music Technology Group (MTG) at Universitat Pompeu Fabra and MusicBrainz, launched in 2014, to crowdsource audio features for millions of music recordings. It leveraged the Essentia library to analyse user-submitted audio fingerprints and collect features. The goal was to enable researchers to have a standardized set for descriptors for MIR tasks, including recommendation systems (Porter et al., 2015).

While the project shut down in 2022, its datasets are still available and cover more than 29 million user submitted records corresponding to 7.5 million unique audio recordings. Each datapoint includes low-level features (spectral features, rhythmic features), high-level features (moods, timbre, etc.) and metadata linked to MusicBrainz identifiers (artist, release year, genre tags).

The AcousticBrainz is an excellent candidate for content-based filtering because of the richness of its data and its diversity (tracks from the 1920s to 2020s) and aligns well with TasteMender's goals.

⁶ <https://developer.spotify.com/documentation/web-api/concepts/rate-limits>

Conclusion

For a stateless music recommendation system, content-based filtering (CBF) emerges as the most suitable foundation. It doesn't require persistent user profiles and can operate effectively on a per-session basis.

In practice, CBF methods represent metadata and features in vectors space and similarity can be determined by measuring cosine distance. This approach has been widely applied in MIR research, from lyrics analysis using TF-IDF to comparing audio features. It offers transparency in how results are determined which aligns with the project goals.

We've investigated how to get data that could drive a recommendation system. Popular MIR datasets have been shown to either be outdated or too limited in scope while APIs are inherently restricted by their providers. Among the available options, AcousticBrainz stands out as the most practical choice and its precomputed audio features could map well to CBF methods.

Recent advances in deep learning have been shown to produce more accurate results, but their implementation comes with significant computational and storage costs. In order to drive a real-time recommendation system while not tanking performance, we could rely on audio features extracted by deep-learning algorithms such as those generated by the Essentia library.

Design (1949/2000 words)

Domain and Users

This project is based upon research from the field of music information retrieval (MIR) and recommender systems. Like commercial streaming services such as Spotify or YouTube Music it uses metadata and audio features to determine song similarities and make recommendations, but it doesn't rely on user data to do so and instead takes a content-driven approach. This method tackles multiple important issues:

1. It ensures user privacy, data is not collected about the users, content and per-session filters dictate the outcomes.
2. It eliminates preference pollution, the outcome is always determined by the input, all sessions are temporary and can be easily reset.
3. It provides transparency, users can easily determine why a certain song was recommended by looking at the filters they set and the API responses.

The TasteMender system provides these benefits because it is designed to operate as a stateless API. Its target audiences are:

1. *Developers* who build music focused front-end applications by integrating the API in their projects and consuming its data. Thus, the data provided needs to be robust enough to support this use case.
2. *Researchers* in the MIR field who wish to explore the underlying dataset and use the system as an experimental platform. The data will need to be complete and accurate.
3. *End users* who are looking to expand their music tastes, explore new content and want explicit control over what's being recommended, in a way which puts privacy first. Thus, a front-end will be required and the recommendations need to be meaningful, varied, and easily interpretable.

These audiences informed the design choices: a stateless API for developers, robust dataset handling for researchers, and a simple but usable frontend for end users.

System Architecture

The technical structure is informed by the target audiences and was designed around a modular three-tier architecture, each tier focusing on a specific user need. The data layer ensures completeness and correctness for researchers, the application layer provides robustness for developers, and the presentation layer allows end users to interact with the system.

Data Layer

The AcousticBrainz dataset contains over 7 million records, including both metadata and extracted audio features. The dataset offers 39 GB of high-level features and 589 GB of low-level features, but only the high-level features were included. Excluding low-level descriptors is a deliberate trade-off: it reduces accuracy slightly but keeps database build times and storage requirements within manageable limits.

High-level features come in the form of numerical predictions made by multiple classifiers for how much a track fits a certain criterion (e.g., a track may be 92.3% danceable). They naturally lend themselves to being stored in a vectorial format for the purpose of similarity comparison. Thus, the data storage strategy is to split the dataset into two parts:

1. Metadata (title, artist, album, genre, release year), mostly needed for presentation purposes, is stored in a PostgreSQL database.
2. Audio features are stored as a vector file and loaded into memory at start-up. This avoids the inefficiency of storing vector data in relational tables and supports fast retrieval during recommendation.

All data is linked by MusicBrainz identifiers (MBIDs) which helps ensure consistency and prevents duplication. A track in the SQL DB can be easily linked to its audio features using this identifier.

Database Design

The metadata is normalized during the ingest process into separate tables for Track, Artist and Album. Many-to-Many relationships are modelled as separate link tables: TrackArtist and AlbumArtist. This structure is inspired by the MusicBrainz database (MusicBrainz, n.d.), from which the metadata was originally derived.

Data which is both presentational and useful for comparing tracks is duplicated between the SQL DB and the in-memory store. This is comprised of: genre classifications (genre_dortmund, genre_rosamerica), number of submissions (duplicates) for determining popularity and album release date. The resulting schema is illustrated in Figure 1 - Entity Relationship Diagram.

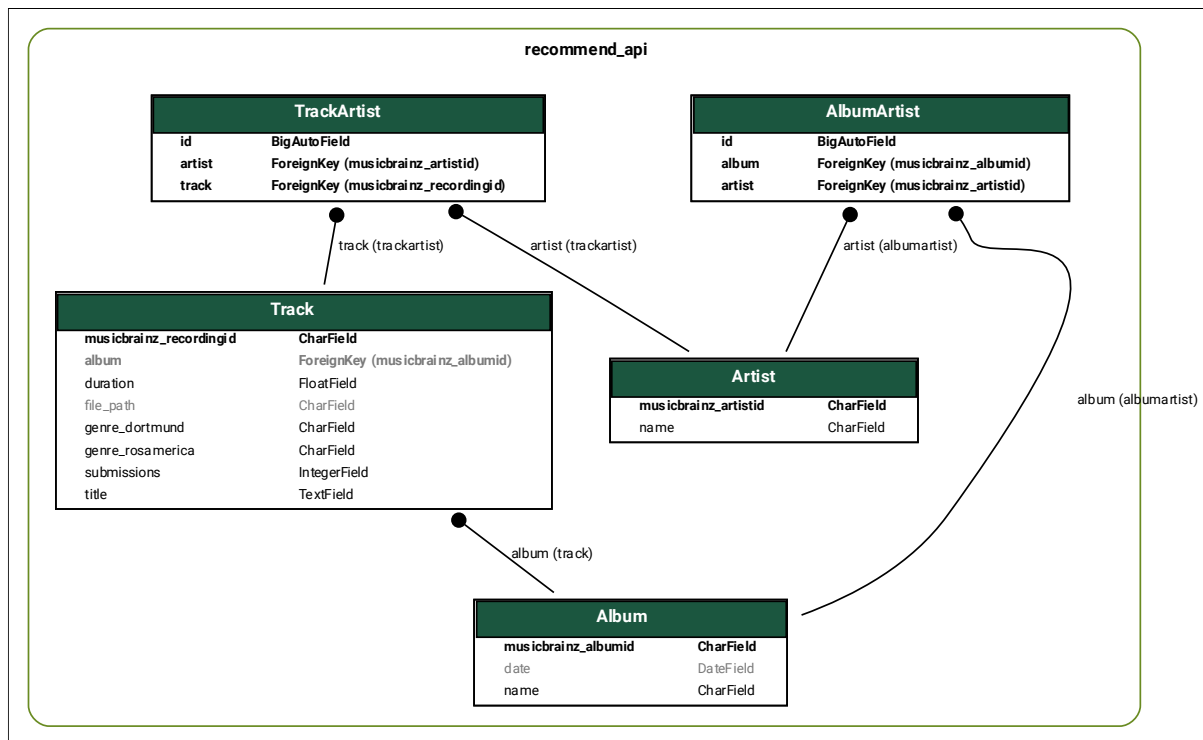


Figure 1 - Entity Relationship Diagram

Data quality issues are also addressed during ingest and normalisation. Duplicate tracks are identified by their MBID and their data is merged by selecting the most common values across submissions. Tracks without an artist or title after the merge process are dropped while album information is optional. Artist name variants were resolved to their most frequent form.

The metadata in the AcousticBrainz dataset was originally derived from the MusicBrainz database. It would have been possible to query this DB directly to get track, artist and album data, but this would introduce a different complication: API rate limits or a new ingest process to consume the MB data dumps. By merging the metadata instead, we keep the ingest process lean.

Application Layer

The application layer acts as a bridge between the data layer and external consumers. It is implemented in the form of a Django project which serves an API through the Django REST Framework (DRF).

All endpoints follow the same design structure:

1. Top level routes return paginated lists of objects (e.g. `/tracks/`)
2. Detail routes return information about a single instance (e.g. `/tracks/<mbid>/`)
3. Nested detail routes return related resources (e.g. `/albums/<mbid>/art/` for cover art urls)

Resource endpoints (detail routes) adhere to the HATEOAS⁷ constraint by including a “links” field which lists the available nested routes (e.g. link to cover art route from album). This makes the REST interface self-describing through its responses. Clients can immediately see what endpoints are available without referring to any documentation or formal schema.

The API exposes endpoints for browsing tracks, artists, and albums, retrieving metadata and features, generating recommendations, and accessing related resources such as cover art and YouTube sources (see Appendix A.2 - API Endpoints). They support only the GET HTTP verb (with one exception), which aligns with the statelessness goals of the project.

The recommendation endpoint (**POST /api/v1/recommend/**) is the only one that accepts POST requests. Filters can be set in the request body to control recommendation logic (Figure 2).

```
{
  // target track that we compare against others
  "mbid": "mbid",
  // listened previously, excluded from results
  "listened_mbids": ["mbid","mbid","mbid"],
  "filters": {
    // if the user "dislikes" a song, we can exclude the artist from future recommendations
    "exclude_artists": ["mbid"],
    "same_genre": true,
    "same_decade": true,
    "genre_classification": "rosamerica"
  },
  // how much each audio feature should impact the similarity score
  "feature_weights": {
    "danceability": 0.5,
    "aggressiveness": 0.5
  },
  // how much similarity score should count vs track popularity in final scoring
  "total_weights": {
    "similarity": 0.7,
    "popularity": 0.3
  },
  // how many results to return
  "limit": 10
}
```

Figure 2 - POST request body for recommend endpoint

The underlying recommendation logic functions as a linear pipeline:

1. Candidate tracks are pre-filtered by genre and decade to match the target track (same_genre and same_decade filters)

⁷ Hypermedia as the engine of application state

2. Previously listened tracks (listened_mbids) are excluded from candidates list
3. Similarity is determined by calculating cosine distance between the feature vectors of the target track and all other candidates
4. Statistical measures are computed: mean, standard deviation, p95, number of valid candidates, search time
5. Candidates are sorted by the cosine distance (lowest first)
6. Weights are applied (feature_weights, total_weights) to re-rank the candidates
7. The candidates are serialized and returned as JSON (Figure 3)

```
{
  "target_track": {
    "mbid": "62c2e20a-559e-422f-a44c-9afa7882f0c4",
    "title": "Enter Sandman"
  },
  "similar_list": [
    {
      "mbid": "9420c245-10aa-43bf-a583-08f0219e5666",
      "title": "Don't Tread on Me",
      "similarity": 0.9326974749565125
    }
  ],
  "stats": {
    "candidate_count": 79448,
    "search_time": 0.008000850677490234,
    "mean": 0.4038538336753845,
    "std": 0.3162822425365448,
    "p95": 0.8052042126655579,
    "max": 0.9930822849273682
  }
}
```

Figure 3 - Recommend endpoint simplified response

To ensure accessibility for developers and researchers, the API is documented using drf-spectatular which generates an OpenAPI schema and provides both swagger and Redoc interfaces.

The application layer reflects the project's three audiences: developers benefit from a documented interface, researchers gain access to rich queries for experimentation, and end users are served privacy-friendly recommendations which they can customize by setting filters and weights.

Presentation Layer

The purpose of the presentation layer is to demonstrate how the API can be consumed in a real-world environment and to server as a demo interface for user testing. It is purposely built as a single page application (SPA) and not tied into Django's template

system. Decoupling the interface means it can be hosted anywhere, having to rely on the API's server only for making requests.

The front-end is build using Vite, a build tool that can spin up applications from predefined templates and which manages all aspects of running a development server and compiling the JavaScript code. React with TypeScript was chosen as the framework since it's the de-facto choice due to its popularity for building SPAs.

The resulting app only needs to provide a few simple bits of functionality:

1. Users can search for tracks by querying the “search” endpoint
2. Playable streams for tracks are provided through the “sources” endpoint
3. Metadata can be displayed by querying the HATEOAS links for each track
4. Recommendations can be requested for any track MBID and should be displayed to the user
5. The user can set filter values for the “recommend” endpoint request in the interface

Prototype and Design Evolution

A prototype was developed during the first phase of development and the lessons learned from it served to inform the current design. We will briefly cover it to justify some of the design decisions mentioned previously.

The prototype used a dataset from Kaggle containing 57k song lyrics (“Spotify Million Song Dataset”). The lyrics were mapped to sparse frequency vectors using Term Frequency-Inverse Document Frequency (TF-IDF) and later compared using cosine similarity (Salton & Buckley, 1988).

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

# generate a matrix of word vectors for each term in the input corpus
vectorizer = TfidfVectorizer(stop_words='english', max_features=10000)
tfidf_matrix = vectorizer.fit_transform(input_corpus)

# for a certain song (song_id), find 10 songs similar to it
query_vec = tfidf_matrix[song_id].reshape(1, -1)
similarities = cosine_similarity(query_vec, tfidf_matrix).flatten()
top_indices = np.argsort(similarities, -11)[-11:]
```

Figure 4 - Prototype feature extraction and similarity

The resulting vectors, limited to 10000 features, were sparse, meaning that most of their values were 0 because each song used a small subset of words from the total domain.

Comparing them took approximatively 2-3 seconds which was too slow for use in a real-time application. These findings informed the use of high-level features in the current project which can be stored in 16 dimensional vectors (acousticness, danceability, etc.) instead of the higher-dimensional low-level features which would have significantly impacted performance.

Looking at the recommendations themselves, songs that emphasize a singular word like Metallica's "Master of Puppets" will be matched to other songs that heavily contain the same word. For this song some of the results are very accurate (Nirvana, Black Sabbath, etc.) which shows a preference for certain words in the rock genre, while outliers highlight the limitations of the approach (Figure 5).

	artist	song	text
12817	Metallica	Master Of Disaster	you should run away when you see me play im t...
45821	Nightwish	Wishmaster	master apprentice heartborne 7th seeker war...
30538	Dream Theater	In Presence Of Enemies Pt. 2	welcome tired pilgrim into the circle we hav...
29546	Depeche Mode	Master And Servant	theres a new game we like to play you see a ...
46024	Nirvana	Downer	portray sincerity act out of loyalty defend ...
30298	Doobie Brothers	The Master	just dont know why i keep on tryin must be a ...
29305	Def Leppard	Answer To The Master	when the night time unfolds and the memories t...
25613	Black Sabbath	Master Of Insanity	look all around cant you open your eyes voic...
15079	Oscar Hammerstein	My Lord And Master	spoken the king is pleased he is pleased with...

Figure 5 - Lyrics based recommendations for "Master of Puppets" by Metallica

Lyrics could be seen as metadata that could constrain results but not determine them. This is why in the current project; audio features were chosen as the basis for determining song similarity while metadata like genre or release year are used as filters.

Project name

The name of the project, TasteMender was chosen as a play on words to highlight some aspects of its purpose:

- "Taste" as in informing musical tastes
- "Mender" because it is a recommender system but also as in "to mend" one's tastes by giving a different kind of recommendation
- "Taste Mender" because combined it sounds like tastemaker

Work Plan

The development was planned for June-September 2025. Tasks were grouped into stages as a natural progression from exploring the domain to delivering the final version. The gaps between some stages are explained by time needed to complete tasks in other

modules. A simplified chart of when each phase took place is listed below (Figure 6) while the full chart including task names is in the appendix (Appendix A.1 - Gantt Chart).

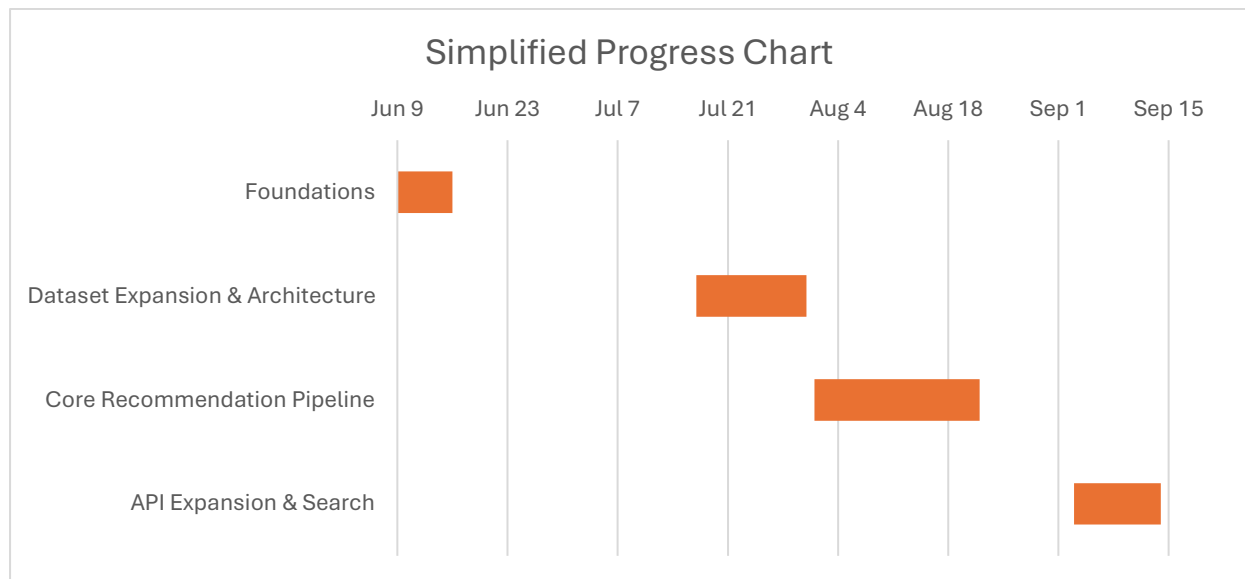


Figure 6 - Graph of the time spanned by each phase

Each phase had specific deliverables and was estimated at taking 14 days to complete including buffer time

1. “Foundations” took 7 days to complete, the target was preparing the prototype and delivering the preliminary report
2. “Dataset Expansion & Architecture” took 14 days, target was ingesting and modelling the AcousticBrainz dataset
3. “Core Recommendation Pipeline” took 21 days, target was building the recommendation engine and serving it as a console command
4. “API Expansion & Search” took 11 days, target was serving recommendations, querying the DB for metadata and searching the DB all through the API

Evaluation plan

Unit tests validate that the API returns a predictable result. They’re also used to recommendation logic: that tracks with similar features have a high similarity score. The data ingest process is also tested to ensure the correctness of the data since manually inspecting the database isn’t feasible due to the large number of records.

The quality of the recommendations is judged by selecting a well-known track (e.g. Metallica’s “Enter Sandman”) and checking that the results correspond both in genre (e.g. rock) but also in sound profile (e.g. does this other track sound similar). The unit tests are still responsible for the mathematical accuracy of the results.

User testing was done in the form of interviews with 5 participants. The users were asked to search for a song, play it, play the next 7 recommendations that come up and judge their quality. Users appreciated the variety of the results.

Implementation (2121/2500 words)

Overview

The implementation is divided into two parts: backend and frontend. The backend is a Django 5.2 project that's responsible for managing the database through its Object-Relational Mapping (ORM) layer and for exposing the recommendation logic through a REST API. The frontend is implemented as a React single-page application SPA. It is intentionally kept separate from the backend to illustrate how an external client can consume the API without requiring tight coupling to the server code.

In the following sections we'll cover some key aspects of the implementation while the full codebase is available on GitHub (<https://github.com/RadValentin/CM3070-FP-Music-Recommendation>). The project's structure is listed below (Figure 7) as a reference for discussing certain files and directories going forward.

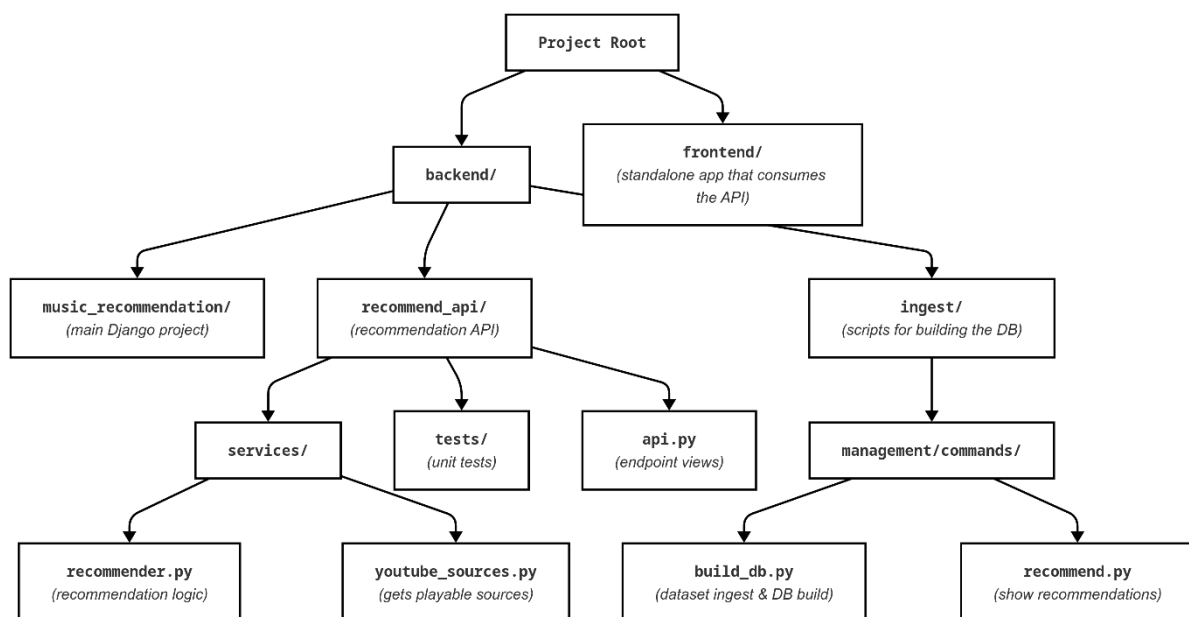


Figure 7 - Code repository structure

Data Ingest Pipeline

Schema

The AcousticBrainz dataset is comprised of 29 million submissions crowdsourced from users of the platform that ran the Essentia audio analyser on their personal music collections. Each submission is offered in JSON format and contains both high-level features (categorical predictions) and low-level features (raw numeric acoustic

descriptors). The data was gathered between 2015 to 2022 and is available in the form of database dumps (**.tar.zst** archives).

This project uses the high-level features to generate recommendations. A sample file is available in Appendix A.3 - AcousticBrainz High-Level JSON for reference. Broadly speaking, each file contains the following:

- Genre classification by different algorithms (**genre_dortmund**, **genre_electronic**, **genre_rosamerica**, **genre_tzanetakis**, **ismir04_rhythm**)
- Moods (**mood_acoustic**, **mood_aggressive**, **mood_electronic**, etc.)
- Audio descriptors (**danceability**, **timbre**, **voice_instrumental**, **tonal_atonal**, **gender**)
- Metadata: recording id, recording title, artist name, artist id, album name, album id, album release date

Each feature has a predicted category (e.g. **value: relaxed**), a probability for that prediction (confidence score) and a distribution across all categories.

File Loading

The ingestion process was implemented in **backend/ingest/pipeline.py** and can be run as a Django command: **python manage.py build_db**. It can read data either by streaming directly from the dump archives or by scanning the directories for JSON files if the archives aren't present.

The archives are processed in parallel using Python's **ThreadPoolExecutor** to distribute tasks across available CPU cores. A 17% boost in read speed was achieved by using an optimized JSON library (orjson) instead of the default one. For 100k files, parsing and processing takes ~37s instead of minutes due to these optimizations.

Data Extraction

A quick look at the contents of the files reveals inconsistencies in the data which motivates the need to process, clean and consolidate the information:

- Release dates are inconsistently formatted, being given as: only the year, in ISO data format, in SQL Datetime format, in ISO 8601 with Zulu (UTC), etc.
- Popular songs by major artists (Nirvana, Beatles) appear frequently with many duplicates.
- Tracks have missing or placeholder titles, ex: [untitled], End Credits, Finale, [unknown]
- Artists appear under multiple names, if two artists collaborate on a track they can appear as: "Jay-Z", "Beyonce" or as a single string "Jay-Z & Beyonce"

JSON data is parsed into Python dictionaries with the help of utility functions defined in **track_processing_helpers.py**. These ensure that each datapoint contains normalized pieces of information.

Duplicate Handling

Popular songs that appear multiple times across user submissions, with slightly different metadata and features, need to be consolidated by grouping them based on MBID and merging into a single entry. The logic depends on the type of information:

1. Numeric features (audio features) are aggregated by the median value
2. Categorical fields are aggregated by selecting the most common non-null value
3. Album information is merged by selecting the most frequent album id + name combination and the median for the release date
4. Artist information is merged by choosing the most common combination of tuples of (**artist_id**, **artist_name**)
5. Distributions (**moods_mirex**) are averaged across duplicates, then re-normalized to sum to 1.

The merging process attempts to extract as much information as possible from the dataset while maintaining correctness through a majority-vote. For example, from 5M submissions:

- 2,615,660 duplicates were identified and merged
- 69,696 files had missing data and were dropped
- 343,937 files had invalid or missing release dates but were kept
- 393,189 files didn't include artist information and were dropped
- 1,921,455 valid unique tracks were retained for database insertion

Database Construction

Metadata is stored in Django ORM models for Track, Artist and Album with two join tables used to model M2M relationships (TrackArtist and AlbumArtist). The unique records are inserted using **bulk_create** operations with batch sizes of 20,000 to minimize ORM overhead.

Audio features are exported separately into a NumPy matrix (**features_and_index.npz**) to support fast comparisons. We'll refer to it as the *feature matrix* going forward. Each row represents one track and contains:

- MBID
- Release year
- Genre labels (Dortmund, Rosamerica)

- 11 continuous features (danceability, aggressiveness, happiness, sadness, relaxedness, partyness, acousticness, electronicness, instrumentalness, tonality, brightness)
- 5-dimensional **moods_mirex** vector

The feature matrix is standardized with StandardScaler and L2-normalized row-wise before export to ensure that any later computations are scale-invariant. The resulting file (~111 MB compressed for 1M tracks) can be loaded directly into memory.

Performance metrics

Below we present some performance metrics to illustrate how the ingest process scales with dataset size on an average test machine (Intel i5-4690K, 16 GB RAM, SATA SSD).

Number of records	100,000	1,000,000	5,000,000
Load JSON files (s)	37.15s	306.32s	1401.32s
Insert artists and albums (s)	0.84s	4.11s	18.64s
Insert records (s)	16.93s	127.57s	470.67s
Insert M2M pairings (s)	15.27s	59.96s	506.50s
Export feature matrix (s)	0.82s	6.26s	27.85s

Table 1 - Performance vs dataset size

Recommendation Engine

The recommendation engine is built on the principle of content-based similarity. Each track is represented as a vector in a high-dimensional space. Audio features make up the vector's values and proximity indicates similarity.

Feature Selection

When considering which features to use for making comparisons we can refer to the accuracy scores published by the AcousticBrainz project:

<https://acousticbrainz.org/datasets/accuracy>. Eleven continuous descriptors stood out due to their high accuracy scores (Table 2) and were chosen as candidates.

Feature name in dataset	Feature name in project	Accuracy
danceability	danceability	92.41%
mood_aggressive	aggressiveness	97.50%
mood_happy	happiness	83.27%
mood_sad	sadness	87.83%
mood_relaxed	relaxedness	93.20%
mood_party	partyness	88.38%
mood_acoustic	acousticness	92.98%
mood_electronic	electronicness	86.38%
voice_instrumental	instrumentalness	93.80%
tonal_atonal	tonality	97.67%

timbre	brightness	94.32%
--------	------------	--------

Table 2 - Features selected and their accuracy scores

Most of the remaining features were disregarded due to the following:

- Genre classifiers are either too inaccurate (genre_dortmund, genre_tzanetakis, ismir04_rhythm, accuracy 60-75%) or too specialized on a single subgenre of music (genre_electronic)
- Gender didn't seem like a relevant way of determining similarity

The Rosamerica genre classification (Guaus, 2009) is better suited as guardrail rather than a feature in cosine similarity space. This is because categorical outputs don't map well to an algorithm that works best with smooth, continuous values. If included directly, the results would collapse into clusters around broad genre centroids, which hides the finer musical differences that matter for recommendations.

The MIREX audio classifier (moods_mirex) gives continuous probabilities for how well a piece of music fits into 5 mood profiles (Hu & Downie, 2007). It describes the audio by targeting perceptual characteristics rather than trying to match a specific genre like other classifiers. This makes it a good candidate feature despite its low accuracy of 57.09%.

Pre-filtering

Matching similarity by 11 continuous features across the full dataset has a high probability of returning irrelevant matches. To reduce the number of outliers, two pre-filters were introduced:

1. Tracks must have the same genre according to Rosamerica classification
2. Tracks must have been released in the same decade

Similarity Computation

Similarity between a target track and other candidate tracks (pre-filtered) is computed using cosine similarity:

$$\text{sim}(a,b) = \frac{a \cdot b}{||a|| ||b||}$$

Normally the output range is $[-1, 1]$ but because we're working with non-negative values for audio features the scores will be in $[0, 1]$ with values close to 1 indicating very high similarity.

Along with the similarity score we use the number of times a track is present in the dataset (submissions) as a popularity indicator to re-rank results and compute a final score.

$$score = 0.9 * similarity + 0.1 * \ln(1 + submissions)$$

Output

The recommendation pipeline can be run as a console command by specifying a target MBID:

```
python manage.py recommend --mbid 62c2e20a-559e-422f-a44c-9afa7882f0c4
```

To validate the feature selection and whether increasing dimensionality produces better results we'll run the recommender for the 11 continuous features in one run and for 36 features (11 continuous, moods_mirex, ismir04_rhythm, genre_tzanetakis) in another on a dataset of 100k submissions. The target track is "Enter Sandman" by Metallica.

11 features	36 features
Stabbing Westward – Shame	Die Toten Hosen – In Dulci Jubilo
Mika Bomb – Super Sexy Razor Happy Girls	Stabbing Westward – I Don't Believe
Bon Jovi – Woman in Love	Die Toten Hosen – Merry X-Mas Everybody
Die Toten Hosen – In Dulci Jubilo	Guns N' Roses – Don't Damn Me
Guns N' Roses – Bad Obsession	Yo La Tengo – Big Day Coming
Sonic Youth – 100% (LP version)	Die Toten Hosen – Entschuldigung, es tut uns lei
Die Toten Hosen – König der Blinden	Queen – The Hero
Stabbing Westward – I Don't Believe	Die Toten Hosen – Weihnachtsmann vom Dach
Melt-Banana – Cannot	Die Toten Hosen – Unsterblich
Guns N' Roses – Don't Damn Me	Stabbing Westward – Crushing Me

Table 3 - How dimensionality affects results

Increasing dimensionality leads to an over-representation of a single artist in the results, "Die Toten Hosen". This confirms the previous assumption that using classifiers (e.g. genre_tzanetakis) as features leads to clustering around certain values, thus introducing noise in the output.

During testing there was no difference between 11 features and 16 features (11 + moods_mirex) on the 100k dataset. We'll use a 2 million dataset to see if how this classifier impacts results (Table 4).

11 features	16 features
Anti-Flag – We've Got His Gun	The Smashing Pumpkins – Appels + Oranjes
The Smashing Pumpkins – Appels + Oranjes	Demoniac – So Bar Gar
Kitchens of Distinction – Cowboys and Aliens	Isengard – Total Death
Happy Drivers – I Shot Da Sheriff (live)	Ария – Бесы

Dark Funeral – Slava Satan	Kitchens of Distinction – Cowboys and Aliens
Ария – Бесы	Abscess – Die Pig Die
Demoniac – So Bar Gar	D-A-D – Blood In/Out
Isengard – Total Death	Dimple Minds – Die Besten Trinken Aus
Abscess – Die Pig Die	Happy Drivers – I Shot Da Sheriff (live)
D-A-D – Blood In/Out	Comecon – The Mule

Table 4 - How MIREX features affect results

It's clear that the MIREX features introduce more signal than noise and are worth keeping. Outliers like Kitchens of Distinction (dream pop / indie rock) and Happy Drivers (psychobilly/rockabilly) are pushed toward the bottom of the list while the top spots are bands like Demoniac, Isengard, Ария, Abscess, all firmly in the heavy/black/death metal spectrum, musically close to Metallica.

API Layer

The API is implemented as a Django app under `recommend_api`. Models for tracks, artists, albums are defined using Django's ORM and linked together using join tables.

Django REST Framework (DRF) provides `ModelViewSet` classes which are used to map DB data to API endpoints, the output being run through serializers first. These classes create list and detail endpoints by default and additional ones can be added by using the `@action` decorator. Pagination and filters are also provided by this framework.

```
class AlbumViewSet(viewsets.ReadOnlyModelViewSet):
    serializer_class = AlbumSerializer
    queryset = Album.objects.prefetch_related("artists")
    lookup_field = "musicbrainz_albumid"
    lookup_url_kwarg = "mbid"
    filter_backends = [OrderingFilter]
    ordering_fields = ["name", "date"]
    ordering = ["pk"]

    @extend_schema(...)
    def retrieve(self, request, *args, **kwargs): ...

    @extend_schema(...)
    @action(detail=True, methods=["get"], url_path="art")
    def art(self, request, *args, **kwargs):
        mbid = self.get_object().musicbrainz_albumid
        response = HttpResponseRedirect(f"https://coverartarchive.org/release/{mbid}/front-250")
        response["Cache-Control"] = "public, max-age=2592000, immutable"
        return response
```

Figure 8 - Album view with "art" action

The API follows RESTful conventions:

- Top-level endpoints return paginated lists (`/api/v1/tracks/`)

- Detail endpoints return single resources (`/api/v1/tracks/<mbid>/`)
- Nested endpoints return related resources (`/api/v1/artists/<mbid>/tracks/`)

All resource endpoints also include HATEOAS-style links in their responses so clients can immediately see what sub-routes are available without consulting documentation.

API Documentation

The project uses **drf-spectacular** to generate documentation dynamically without the need for a separate build step. It supports OpenAPI 3.0 schema, Swagger and Redoc. The views it provides (e.g. **SpectacularAPIView**) map to API endpoints and are listed on the API root.

The `@extend_schema` decorator is applied to ViewSet methods to specify additional info that will show up in the documentation including:

- Choice of serializer for regular APIViews (responses)
- Request format serializer (request)
- Text description (description)
- Query string parameters (parameters)

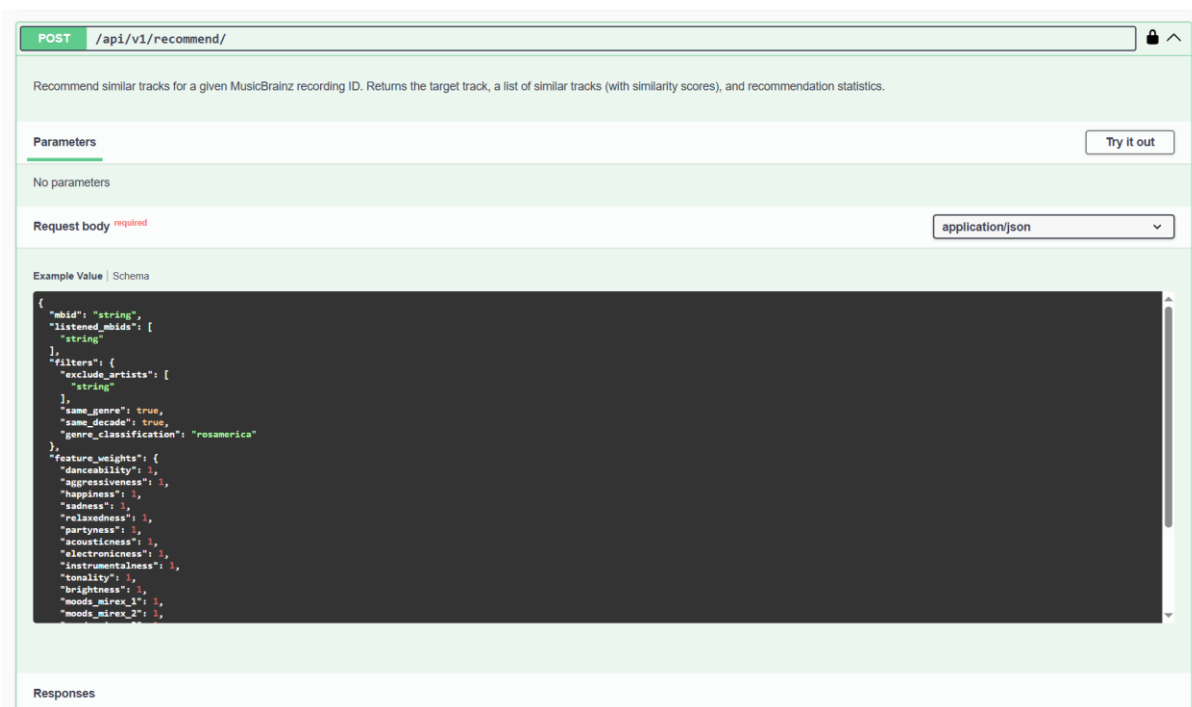


Figure 9 - Swagger docs for recommend endpoint

Search

The search endpoint uses trigram similarity to compare how many overlapping 3-character substrings (or trigrams) two strings share. Two strings with many common trigrams are considered to be very similar. For example, from the string "Alice" we can

create 3 trigrams: "ali", "lic", and "ice". This method allows for fuzzy matching and tolerates typos or partial inputs.

Trigrams operations are available in PostgreSQL through the `pg_trgm` extension and in Django through the `django.contrib.postgres` app.

```
Track.objects.filter(title__trigram_similar=query)
    .annotate(distance=TrigramDistance("title", query))
    .order_by("distance")

-- The resulting SQL
SELECT title, (title <-> 'sunshine of your love') AS distance
FROM recommend_api_track
WHERE title % 'sunshine of your love'
ORDER BY distance ASC;
```

Figure 10 - Django QuerySet and the SQL it translates to

To keep search performant on a database with 2M rows or more, special indexes were applied to the fields of the models being searched. They reduce average query times from ~0.46s to ~0.03s:

- *GIN trigram index* accelerates filtering operations like `WHERE title % 'q'`
- *GiST trigram index* enables k-NN (nearest-neighbour) ordering for queries like `ORDER BY title <-> 'q'` without scanning/sorting the entire table

```
# Example for track model
class Track(models.Model):
    title = models.TextField()
    class Meta:
        indexes = [
            GinIndex(fields=["title"], name="track_title_trgm", opclasses=["gin_trgm_ops"]),
            GistIndex(fields=["title"], name="track_title_trgm_gist", opclasses=["gist_trgm_ops"]),
        ]
```

Figure 11 - Indexes on Track model

The endpoint is exposed as `GET /api/v1/search/?q=<query>`.

Recommendation Endpoint

The recommendation engine is exposed through a dedicated endpoint: `POST /api/v1/recommend/`. Clients submit a recording ID and optional filters via the request body. The API retrieves the feature matrix from memory, applies pre-filters, computes similarity and returns a ranked list of recommendations.

Front-End Integration

The frontend was bootstrapped with Vite, a widely used build tool, using its React + TypeScript template as a starting point. Communication with the API is handled using Axios, an open-source HTTP client. Custom methods for querying the API are defined in **api.ts**.

In production, the React app is served from the Django project through a view (SPAView) which loads the bundle generated by Vite (npm run build). For development we can use the Vite server to load the front-end (**npm run dev**).

Styles are written in plain CSS and leverage CSS nesting to wrap each component's styles in a namespace, so they don't affect other components.

The root component, App, is responsible for consuming the **top_tracks** endpoint and displaying its data as a homepage. Users submit search queries to its child component, Header, which triggers the parent to load and render search results from the **search** endpoint.

The Player component handles audio playback using a YouTube iframe player. When a user clicks play on a track in the interface, the track object is loaded into the player's state. This triggers the following actions:

1. A playable source (YouTube video ID) is requested for the track through the **sources** endpoint
2. The iframe player loads the source and begins playback
3. A payload is prepared for the recommend endpoint including the target track ID, previously listened songs and filters
4. The **recommend** endpoint is consumed and the returned results are rendered next to the video, indicating which one will be played next

The Filters component is rendered inside the Player. It contains input elements with corresponding values stored in component state. When the user changes the values of filters, the parent component is notified through a callback method and will in-turn re-fetch the recommendations by sending the updated filters to the endpoint.

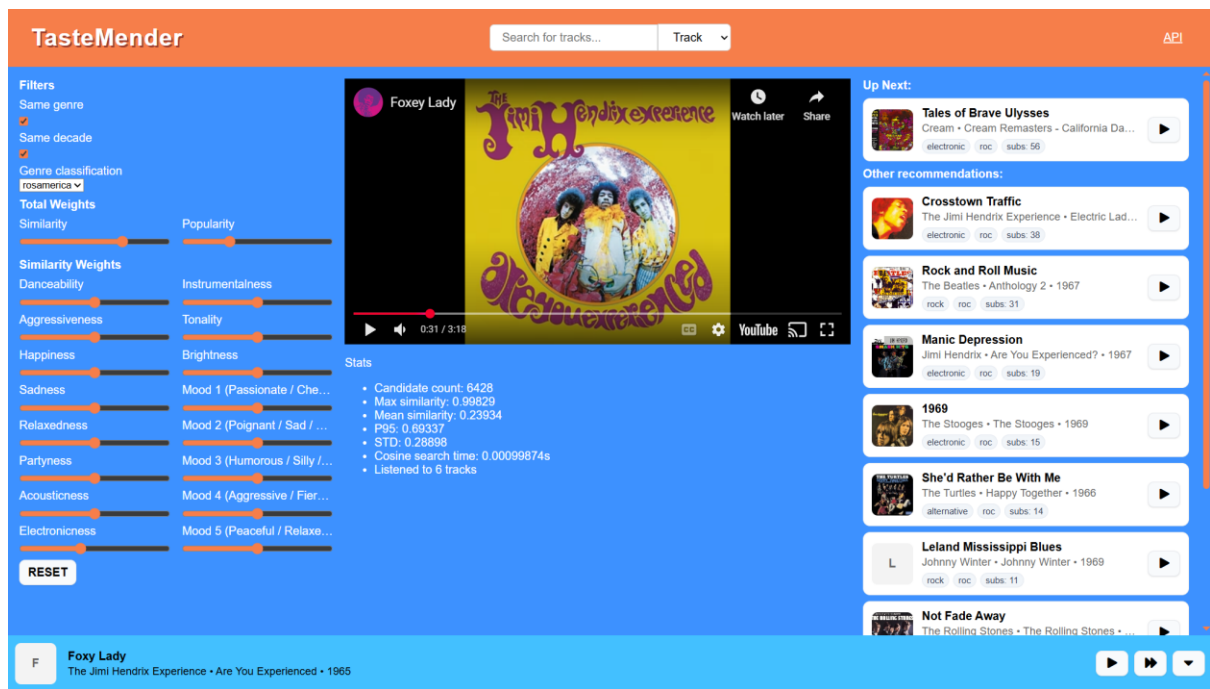


Figure 12 - The player interface

Running the project

To run locally, install Python 3.12, Node.js 20, and PostgreSQL 17, then clone the repo, install backend/frontend dependencies, migrate the database, and start the servers. Full step-by-step instructions are included in the project README and Appendix A.4 - Running the Project.

Evaluation (1151/2500 words)

To evaluate the current project, we'll look at the performance of the system, the quality of the results it generates and its perceived quality from a user perspective. A critical analysis is also included which covers both trade-offs made during the design and implementation phases as well as weaknesses that were identified. A short plan for mitigating shortcomings is included under future work.

Performance Evaluation

The API is designed to be used in real time. The recommendation engine returns a response under 500ms for a DB with 2M tracks while the search endpoint takes between 500ms and 1.4s to return results.

The size of the total dataset made it challenging to do a full ingest (est. time 4-5 hours) during development, so we only worked with a subset of the data. However, the performance of the system should remain within reasonable limits even with the full 7.5M tracks loaded due to genre and decade gates which limit the number of candidates before cosine similarity is computed, and the trigram indexes which drive search queries.

Beyond the response times, additional performance metrics were analysed. The feature matrix occupies 255 MB for 2M tracks, meaning that even for the full dataset only 1GB of RAM would be required for in-memory operation. This is a reasonable memory footprint for modern systems. Regarding disk space, the project files take up less than 500MB and the PostgreSQL would require 9GB for the full dataset. These figures illustrate the feasibility of the project.

Recommendation Quality

Evaluating the quality of recommendations is inherently subjective since it depends on the listeners perspective. From a logical perspective we rely on the quality of the audio features present in the dataset and present similarity scores in the recommendation responses to verify their correctness. Unit tests help ensure that the scores are accurate.

During development quality was judged by selecting well known tracks from major bands and checking if they're matched with tracks that sound similar and are in the same genre. We've looked at how the audio features affect similarity in Table 3 and Table 4 and selected the ones that provide the best results.

Given the large size of the dataset, for any track we can find matches with similarity scores above 0.9, indicating closeness in the feature space. More than 100 tracks were tested during development to validate the scores.

User Testing

To complement the quality evaluation, a small-scale user study was conducted. Five participants, aged between 29 and 38 with diverse musical tastes were asked to interact with the front-end interface and give their thoughts. The study was structured as an interview, and the participants had to complete the following tasks:

1. Search for a track and play it
2. Judge the quality of the recommendations presented
3. Use filters to make changes to what was recommended (if needed)
4. Play one of the recommendations and repeat the process 7 times
5. Provide feedback on whether the results felt musically coherent

The overall findings were that most users enjoyed the discovery aspect of the system. In many cases they were recommended new tracks they hadn't listened to before, especially when turning on the popularity re-ranking filter. Participants also praised the level of control offered by the interface through its various filters.

In terms of limitations, participants noted some outliers present in the results, likely related to the quality of the dataset (e.g. wrong genre labels, incorrect features). The playable sources sometimes could not be identified so tracks wouldn't play.

While the study was small, it did help reveal some key issues that were ignored during development but could serve as ideas for future improvements.

Critical Analysis

The overall strengths of the project are that it produced a performant API that delivers meaningful recommendations without the use of personal data, as demonstrated so far. We'll focus next on the trade-offs that had to be made to achieve this.

The data ingest process while functional is very costly in terms of run times which are determined by the need to de-duplicate submissions. The pipeline tries to normalize both audio features and track metadata. This approach seemed feasible at the start of development but as the project grew it became increasingly apparent that much of this work could be simplified by referring to a canonical source of information.

The MusicBrainz dataset represents a solid option for optimizing the data ingest process. By querying it we could extract accurate titles for tracks, artist and album data without the need to aggregate what's present in the AcousticBrainz files, using them only for extracting audio features. Unfortunately, this aspect became apparent late in development and the cost of consuming a second dataset was deemed too high, an assumption that doesn't hold up anymore.

Another limitation of the system is the quality of search results. Currently search is done on a single database field (title for tracks, name for albums and artists). By

combining multiple fields in the search, users would have a more intuitive way of performing searches. This comes with a performance cost that the current approach based on trigrams isn't well suited to. Other ways of performing full-text search like Django's SearchVector should be explored. For the purposes of music recommendations, search we deemed to be a secondary priority but going forward it could be improved to better cater to user needs.

Dataset Limitations

Inherent limitations in the quality of the AcousticBrainz dataset present challenges that aren't easily addressable. The project was discontinued in 2022 after concerns were raised about the accuracy of its feature extraction process (*AcousticBrainz*, 2022). While it remains one of the most complete openly available resources for MIR, the risk of inaccuracies may reduce recommendation quality as indicated by our previous findings with outliers being present in the results sometimes.

There's no clear path of mitigation at the time of writing, with no other datasets available that cover such a large number of recordings. The project therefore accepts this trade-off: completeness at the expense of guaranteed accuracy.

Future Work

The evaluation revealed several areas of improvement that could be taken into account going forward.

The performance of the system has yet to be tested in a production environment with the full 7.5M tracks dataset loaded. In order to support this, one clear improvement would be the adoption of an optimized similarity algorithm using approximate nearest neighbour (ANN) search, such as FAISS or Annoy. These could replace the current brute-force cosine similarity computation and increase performance tenfold while maintaining high accuracy.

Endpoints, in general, could receive a significant speed boost by implementing a caching strategy. Popular tracks are likely to be queried frequently thus caching responses at the API layer could reduce computation and improve throughput. Different full-text search strategies could be explored for the search endpoint in order to further decrease query times.

The front-end, which was written for demonstration purposes only could be refined into a functional application by adding additional views that expose metadata through artist and album pages.

Conclusion (528/1000 words)

The current project set out to explore whether and how a stateless, privacy-friendly music recommendation system could be delivered through a REST API and demonstrated with a React frontend. The motivation came from concerns regarding heavy profiling and surveillance practices in commercial platforms such as Spotify and YouTube Music. By contrast, TasteMender was designed to leverage content-based filtering and explicit user control over the recommendation process showing that musically coherent results could be generated without needing to collect user data or maintain persistent profiles.

We've shown how the design of the system was informed by the initial motivation as well as the available open data sources. The quality of the data shaped the way in which the design and later implementation evolved. Continuous high-level audio features were demonstrated as a valid way of determining similarities between tracks while categorical ones or metadata introduced more noise than signal and were better suited as a way to restrict the candidates to a certain genre and/or decade subset.

Delivering the metadata and recommendations through a real-time REST API imposed technical challenges both in terms of how the data is ingested into the database and how computations are made. Optimizations like parallel streaming of dataset archives, trigram indexing for search and storing the audio features in RAM were crucial in keeping response times manageable, often under 500ms.

User testing helped to validate the quality of the implementation and the performance of the system. Participants appreciated the musical closeness of the recommendations and the level of control over them while at the same time signalling some issues of the underlying dataset (mismatched or mislabelled tracks). The validation was restricted to a small, demographically narrow group, making results indicative rather than conclusive.

Future work could improve the current project in several ways. Integrating metadata directly from MusicBrainz would reduce the total ingest times and improve data accuracy. Faster search algorithms (FAISS or Annoy) would scale better with heavier user loads. A mature front-end interface that adheres to React best practices and the introduction of richer features such as playlists and recommendation history would improve user experience and engagement.

From a personal perspective, this project deepened my understanding of the challenges faced by data scientists and researchers. Tasks like data cleaning which were previously only encountered as simple exercises became almost deal-breaking challenges at scale. It cannot be understated how noisy crowdsourced data can be, and no amount of mathematical rigour during processing can make up for inherent flaws in the data. I suspect that like the creators of AcousticBrainz, I can never be 100% certain in the

accuracy of the data used to make recommendations. However, the consistent results produced by the system during testing help to alleviate this uncertainty and to highlight the success of this project.

In conclusion, TasteMender achieved its central goal of delivering a recommendation system that produces meaningful results in a performant manner while respecting the privacy of its users. While there are many areas of potential improvement, through this report we've shown that the core functionality is there and can serve as a basis for taking things to the next logical level, the deployment of a production-ready application.

References

- AcousticBrainz: Making a hard decision to end the project* – MetaBrainz Blog. (2022, February 16). <https://blog.metabrainz.org/2022/02/16/acousticbrainz-making-a-hard-decision-to-end-the-project/>
- Anderson, C. (2006). *The long tail: Why the future of business is selling less of more* (1st ed). Hyperion.
- Asad, M., Shaukat, S., Javanmardi, E., Nakazato, J., & Tsukada, M. (2023). A Comprehensive Survey on Privacy-Preserving Techniques in Federated Recommendation Systems. *Applied Sciences*, 13(10), Article 10. <https://doi.org/10.3390/app13106201>
- Bertin-Mahieux, T., Ellis, D. P. W., Whitman, B., & Lamere, P. (2011). *The Million Song Dataset*. <https://doi.org/10.7916/D8NZ8J07>
- Burke, R. (2002). Hybrid Recommender Systems: Survey and Experiments. *User Modeling and User-Adapted Interaction*, 12(4), 331–370. <https://doi.org/10.1023/A:1021240730564>
- Celma, Ò. (2010). *Music recommendation and discovery: The long tail, long fail, and long play in the digital music space*. Springer.
- Choi, K., Fazekas, G., Sandler, M., & Cho, K. (2016). *Convolutional Recurrent Neural Networks for Music Classification* (No. arXiv:1609.04243). arXiv. <https://doi.org/10.48550/arXiv.1609.04243>
- Felgenbaum, E. A. (1977). The art of artificial intelligence: Themes and case studies of knowledge engineering. *Proceedings of the 5th International Joint Conference on Artificial Intelligence - Volume 2*, 1014–1029.
- Gomez-Uribe, C. A., & Hunt, N. (2016). The Netflix Recommender System: Algorithms, Business Value, and Innovation. *ACM Trans. Manage. Inf. Syst.*, 6(4), 13:1-13:19. <https://doi.org/10.1145/2843948>
- Guaus, E. (2009). *Audio content processing for automatic music genre classification: Descriptors, databases, and classifiers*. <https://www.tdx.cat/handle/10803/7559>
- He, X., Liao, L., Zhang, H., Nie, L., Hu, X., & Chua, T.-S. (2017). *Neural Collaborative Filtering* (No. arXiv:1708.05031). arXiv. <https://doi.org/10.48550/arXiv.1708.05031>
- Hu, X., & Downie, J. S. (2007). Exploring Mood Metadata: Relationships with Genre, Artist and Usage Metadata. *ISMIR*, 67–72. https://ismir2007.ismir.net/proceedings/ISMIR2007_p067_hu.pdf?q=mood
- Johnson, J., Douze, M., & Jégou, H. (2021). Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535–547. <https://doi.org/10.1109/TBDDATA.2019.2921572>
- Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix Factorization Techniques for Recommender Systems. *Computer*, 42(8), 30–37. <https://doi.org/10.1109/MC.2009.263>

- List of online music databases. (2025). In *Wikipedia*.
https://en.wikipedia.org/w/index.php?title=List_of_online_music_databases&oldid=1308584794
- Melville, P., & Sindhvani, V. (2011). Recommender systems. In *Encyclopedia of machine learning* (pp. 829–838). Springer.
- Murray, N. (2009, August 26). One Nation, Still More Or Less Under A Groove. *AV Club*.
<https://www.avclub.com/one-nation-still-more-or-less-under-a-groove-1798217490>
- MusicBrainz. (n.d.). *MusicBrainz Database / Schema—MusicBrainz*. MusicBrainz. Retrieved September 18, 2025, from https://musicbrainz.org/doc/MusicBrainz_Database/Schema
- Porter, A., Bogdanov, D., Kaye, R., Tsukanov, R., & Serra, X. (2015). *Acousticbrainz: A community platform for gathering music information obtained from audio*.
<https://repositori.upf.edu/items/0b444dc2-e4d6-4858-a37a-909b6e00a328>
- Qin, Y. (2013). *A historical survey of music recommendation systems: Towards evaluation*. McGill University (Canada).
<https://search.proquest.com/openview/fab3624898343ce179b26d5c2d48d121/1?pq-origsite=gscholar&cbl=18750&diss=y>
- Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5), 513–523. [https://doi.org/10.1016/0306-4573\(88\)90021-0](https://doi.org/10.1016/0306-4573(88)90021-0)
- Salton, G. (with Internet Archive). (1971). *The SMART retrieval system: Experiments in automatic document processing*. Englewood Cliffs ; [Hemel Hempstead] : Prentice-Hall.
<http://archive.org/details/smartretrievalsy0000salt>
- Spotify. (n.d.). *Spotify/annoy* [C++]. Spotify. Retrieved September 16, 2025, from <https://github.com/spotify/annoy> (Original work published 2013)
- van den Oord, A., Dieleman, S., & Schrauwen, B. (2013). Deep content-based music recommendation. *Advances in Neural Information Processing Systems*, 26.
https://papers.nips.cc/paper_files/paper/2013/hash/b3ba8f1bee1238a2f37603d90b58898d-Abstract.html

Appendix

Appendix A.1 - Gantt Chart

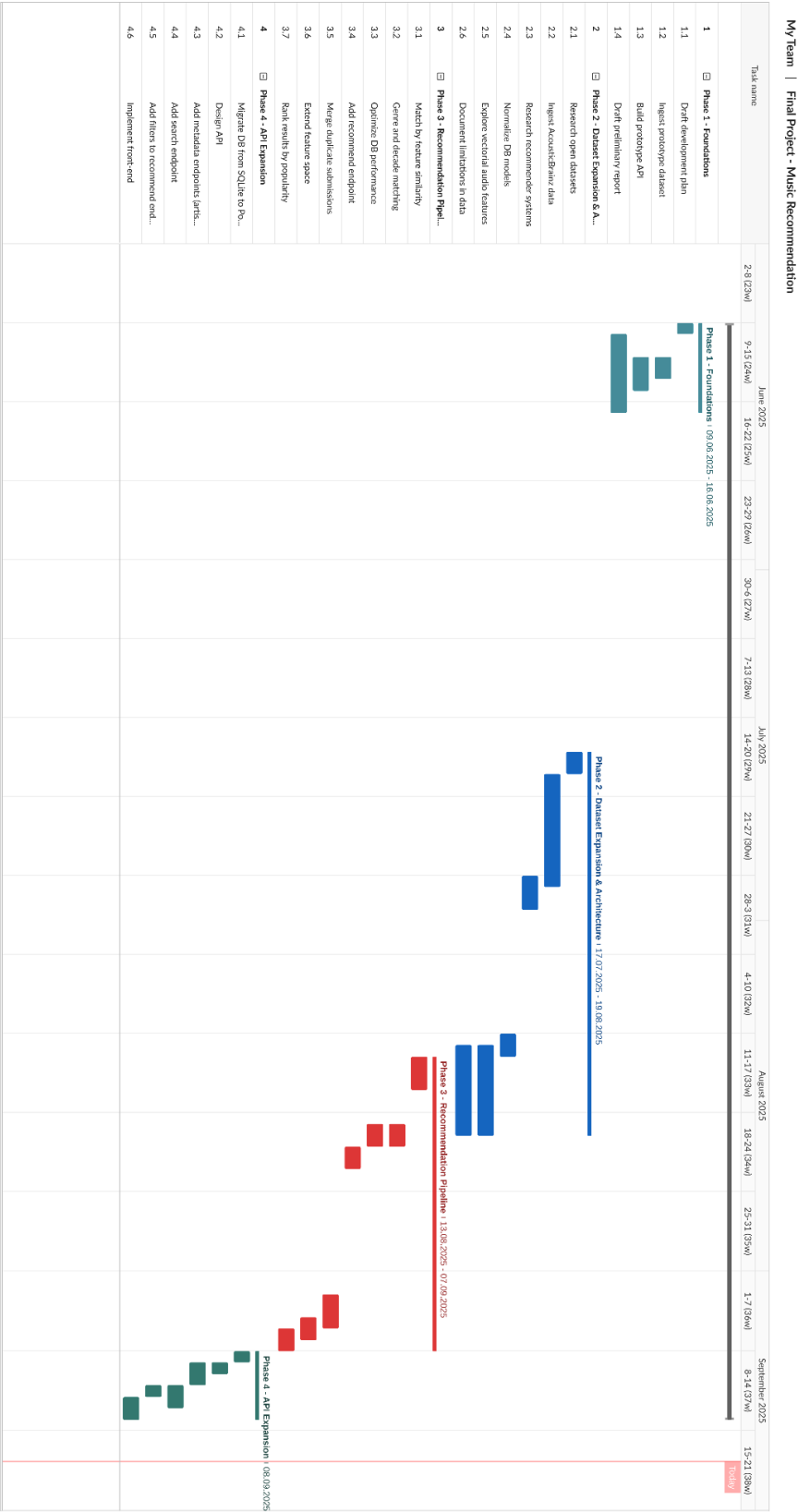


Figure 13 - Gantt Chart for Project Development

Appendix A.2 - API Endpoints

- **GET /api/v1/** - list of available endpoints
- **GET /api/v1/tracks/** - list of tracks (paginated), sortable by title and album date
- **GET /api/v1/tracks/<mbid>/** - details for a single track
- **GET /api/v1/tracks/<mbid>/sources/** - return playable source for a track
- **GET /api/v1/tracks/<mbid>/features/** - show audio features for a track
- **GET /api/v1/artists/** - list of artists (paginated), sortable by name
- **GET /api/v1/artists/<mbid>/** - details about a single artist
- **GET /api/v1/artists/<mbid>/tracks/** - all tracks by artist (paginated)
- **GET /api/v1/artists/<mbid>/top-tracks/** - top tracks by artist (paginated)
- **GET /api/v1/artists/<mbid>/albums/** - albums by artist, sorted by date
- **GET /api/v1/albums/** - list all albums (paginated) sortable by name and date
- **GET /api/v1/albums/<mbid>/** - album details
- **GET /api/v1/albums/<mbid>/art/** - return external URL for album cover art
- **GET /api/v1/genres/** - list all unique genre names in Rosamerica and Dortmund classifications
- **GET /api/v1/search/** - searches for tracks, albums, artists, params are set as query strings
- **POST /api/v1/recommend/** - get recommendations for a song (MBID in POST body)

Appendix A.3 - AcousticBrainz High-Level JSON

Simplified example of a high-level JSON file from the AcousticBrainz dataset, the full contents are available at: <https://acousticbrainz.org/00000baf-9215-483a-8900-93756eaf1cfc/high-level/view?n=0>.

```
{
  "highlevel": {
    "danceability": {
      "all": {
        "danceable": 0.999909281731,
        "not_danceable": 0.0000907100911718
      },
      "probability": 0.999909281731,
      "value": "danceable"
    },
    "gender": {},
    "genre_dortmund": {},
    "genre_electronic": {},
    "genre_rosamerica": {
      "all": {
        "cla": 0.000191239509149,
        "dan": 0.747053861618,
        "hip": 0.0434639304876,
        "jaz": 0.000737536873203,
        "pop": 0.0537411235273,
        "rhy": 0.00738812517375,
        "roc": 0.147098049521,
        "spe": 0.00032613473013
      },
      "probability": 0.747053861618,
      "value": "dan"
    },
    "genre_tzanetakis": {},
    "ismir04_rhythm": {},
    "mood_acoustic": {},
    "mood_aggressive": {},
    "mood_electronic": {},
    "mood_happy": {},
    "mood_party": {},
    "mood_relaxed": {},
    "mood_sad": {},
    "moods_mirex": {
      "all": {
        "Cluster1": 0.352815359831,
        "Cluster2": 0.0591079592705,
        "Cluster3": 0.0789495408535,
        "Cluster4": 0.0582095906138,
        "Cluster5": 0.450917541981
      },
      "probability": 0.450917541981,
      "value": "Cluster5"
    },
    "timbre": {},
    "tonal_atonal": {},
    "voice_instrumental": {}
  },
  "metadata": {
    "tags": {
      "title": ["Como Podem"],
      "artist": ["In Extremo"],
      "album": ["Weckt die Toten!"],
      "date": ["1998-05-01"],
      "musicbrainz_recordingid": ["00000baf-9215-483a-8900-93756eaf1cfc"],
      "musicbrainz_albumid": ["cc7a7e52-9c99-398b-a326-10505a92517e"],
      "musicbrainz_artistid": ["8ebd161e-f45e-41b9-8019-fcbd094c327f"]
    }
  }
}
```

Appendix A.4 - Running the Project

The project requires the following prerequisites to be installed:

- Python 3.12.4 (or newer)
- Node.js v20.17.0 (or newer)
- PostgreSQL 17.6
- Git
- (optional) a Python virtual environment

The source code can be downloaded locally using this command:

```
git clone https://github.com/RadValentin/CM3070-FP-Music-Recommendation.git
cd CM3070-FP-Music-Recommendation
```

AcousticBrainz high-level datasets needed to build the database should be downloaded from the official source: <https://acousticbrainz.org/download>. They come in the form of **.tar.zst** archives split into multiple parts. It is recommended to download *only the sample dataset* for development or demo purposes from this URL:

<https://data.metabrainz.org/pub/musicbrainz/acousticbrainz/dumps/acousticbrainz-sample-json-20220623/acousticbrainz-highlevel-sample-json-20220623-0.tar.zst>

Next, a PostgreSQL user and database need to be set up by running the following commands:

```
CREATE USER django WITH PASSWORD 'password';
CREATE DATABASE taste_mender_db WITH ENCODING 'UTF8' OWNER django;
GRANT ALL PRIVILEGES ON DATABASE taste_mender_db TO django;
GRANT ALL PRIVILEGES ON SCHEMA public TO django;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO django;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO django;
-- Needed for creating a DB when running tests
ALTER USER django CREATEDB;
```

The Django project needs to be configured with the DB credentials, paths to dataset archives, and Youtube API key by creating a **.env** file in the **backend/** directory. For simplicity, an example config file is provided directly in the repository (**.env.example**), its contents are similar to Figure 14.

```
# Paths to where AcousticBrainz dataset files were unzipped.
# Archives can be downloaded from here: https://acousticbrainz.org/download
AB_HIGLEVEL_ROOT=D:/Datasets/AcousticBrainz/High-level
AB_SAMPLE_ROOT=D:/Datasets/AcousticBrainz/Sample

# PostgreSQL settings
POSTGRES_DB=taste_mender_db
POSTGRES_USER=django
POSTGRES_PASSWORD=password
POSTGRES_HOST=127.0.0.1
POSTGRES_PORT=5432

# Django settings
DJANGO_SECRET_KEY="django-secret-key"
DJANGO_DEBUG="True"
DJANGO_ALLOWED_HOSTS=localhost,127.0.0.1

# External API keys
YOUTUBE_API_KEY="youtube-api-key"
```

Figure 14 - Example .env file

Next, we can install the required dependencies for Django, build the database using the sample dataset, and run the server:

```
cd backend/
pip install -r requirements.txt
python manage.py migrate
python manage.py build_db --sample
python manage.py test
python manage.py runserver
```

The server should now be running at <http://localhost:8000/> and we should be able to query the API through DRF's web interface by going to <http://localhost:8000/api/v1/>.

We can also run the front-end by using Vite's development server. It should be available at <http://127.0.0.1:5173> after running the following instructions in a separate console window:

```
cd frontend/
npm install
npm run dev
```